

Contents

- Payments Service — Design Spec
 - Architecture
 - Request flow
 - Flow Sequence
 - State machine
 - State Diagram
 - Data Schema

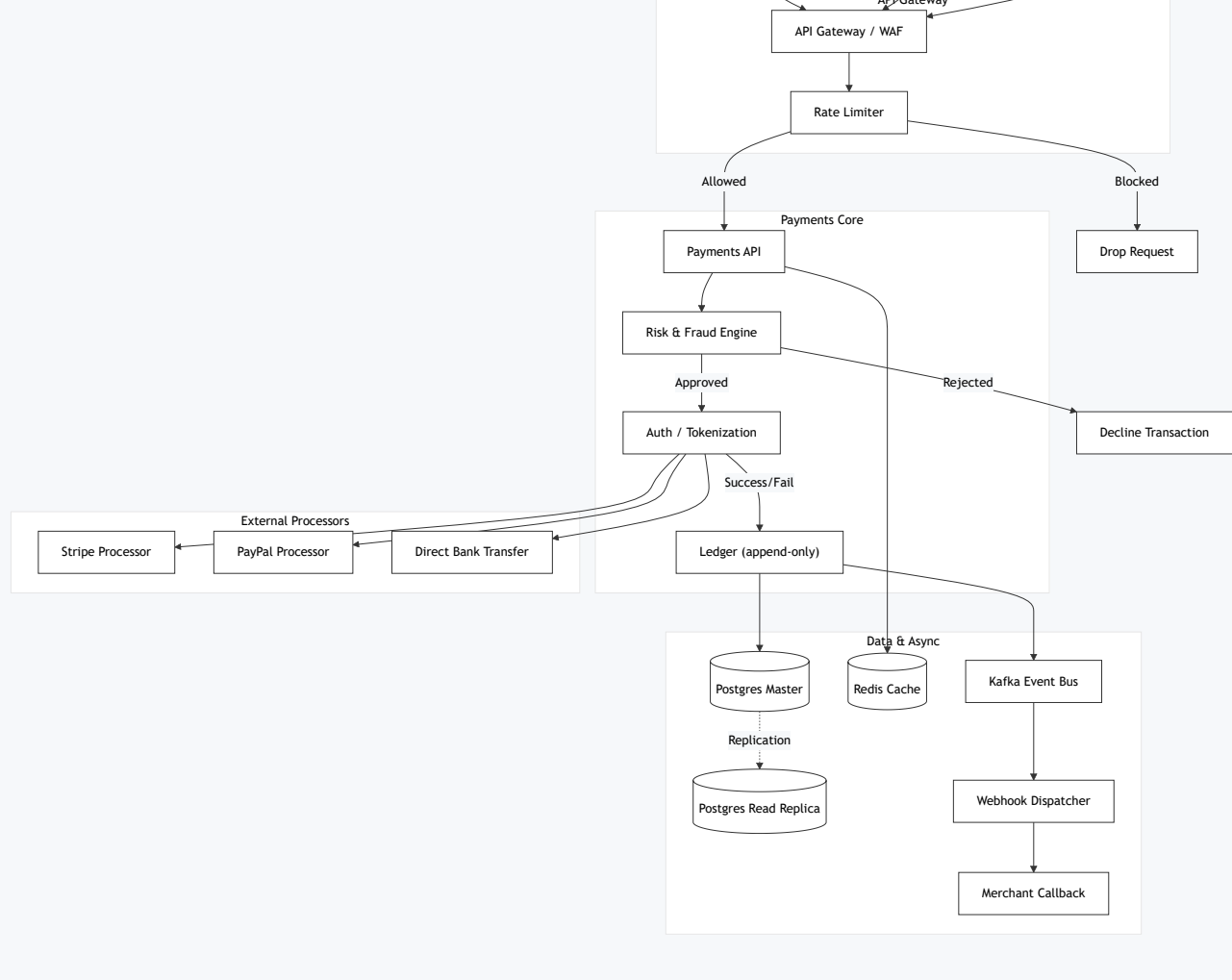
Payments Service — Design Spec

A small, auditable service that authorises card payments and records every state transition.

Note

This document is illustrative. Amounts and identifiers are examples, not production values.

Architecture



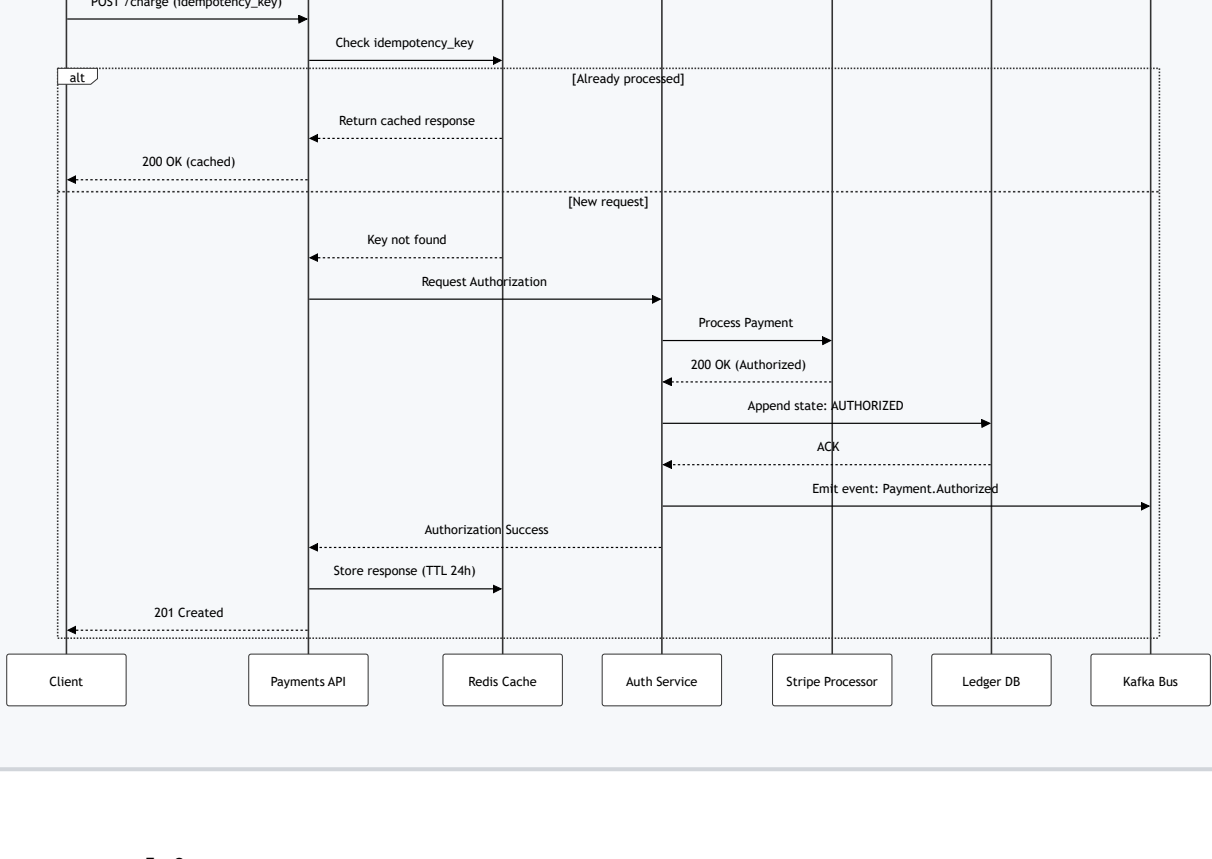
Request flow

1. The client posts a charge with an idempotency key.
2. The API tokenizes the card and calls the processor.
3. Every transition is appended to the **ledger** — nothing is ever mutated in place.
4. A webhook notifies the merchant asynchronously.

```
{
  "idempotency_key": "chg_9f2a...",
  "amount": 4200,
  "currency": "AUD",
  "capture": true
}
```

Flow Sequence

The following sequence details a successful payment capture, including caching, authorization, and asynchronous webhook dispatch:



State machine

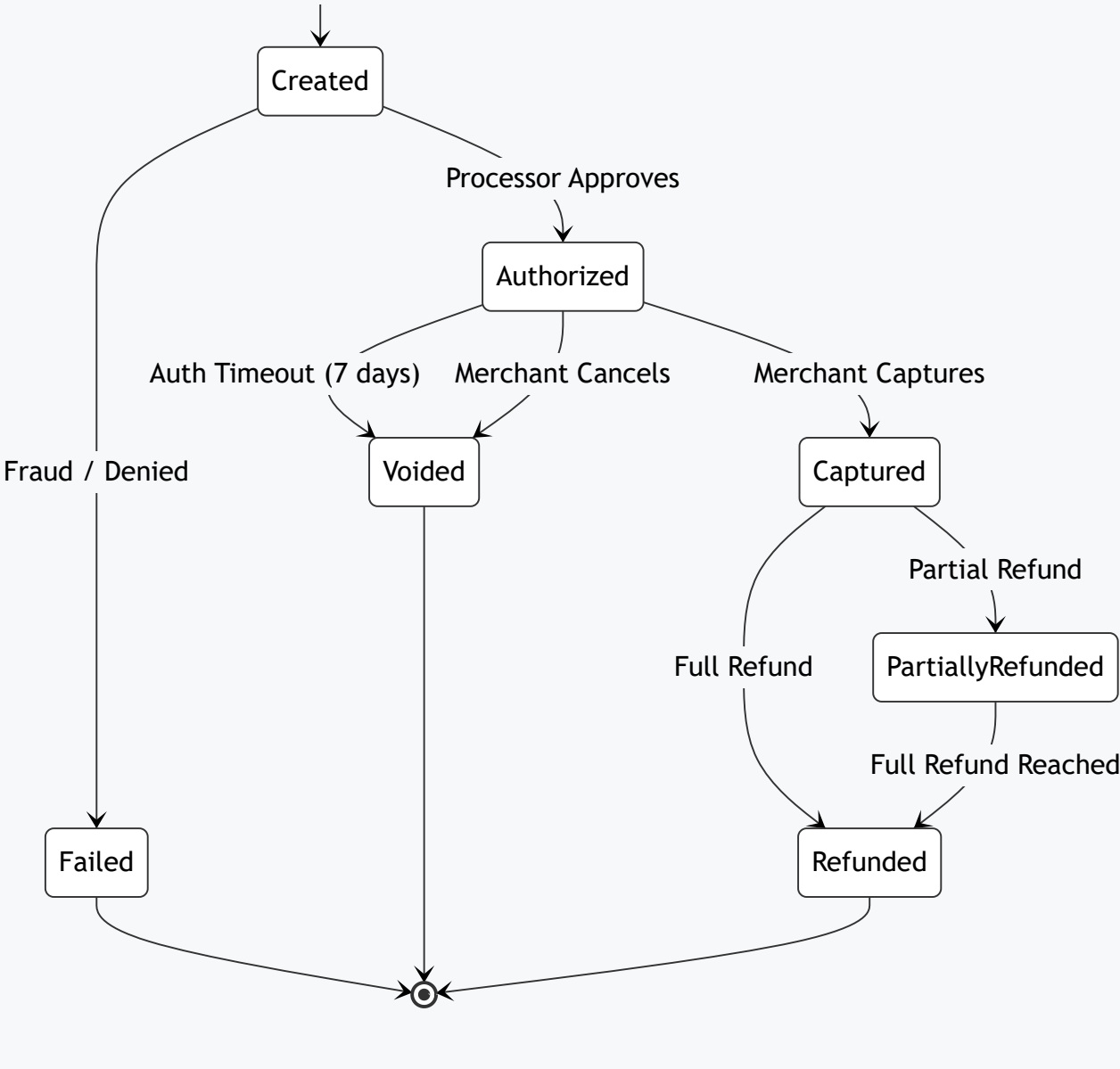
State	Next	Trigger
created	authorized	processor approves
authorized	captured	capture requested
authorized	voided	timeout / cancel
captured	refunded	refund issued

Warning

Idempotency keys expire after 24h. Re-using an expired key starts a **new** charge.

State Diagram

Below is the state transition logic for any given transaction inside the core engine:



Data Schema

The underlying database uses an append-only architecture to maintain the **Ledger**. This requires specific mappings.

MERCHANT		
uuid	id	PK
string	name	
string	webhook_url	
boolean	is_active	

processes

TRANSACTION		
uuid	id	PK
string	idempotency_key	UK
int	amount	
string	currency	
string	status	
timestamp	created_at	

contains

LEDGER_ENTRY		
uuid	id	PK
uuid	transaction_id	FK
string	previous_state	
string	new_state	
timestamp	recorded_at	